



lcart-3 60W 0803

INTEGRATING YP-SPUR AND ROS2
(UBUNTU LINUX 22.04)

What was used

- ▶ I-Cart 3
- ▶ Ubuntu Linux 22.04
- ▶ ROS2 Humble
- ▶ YP-Spur(BLVR mode)
- ▶ Hokuyo URG (LiDAR)
- ▶ Logicoool F710 controller

What needs to be installed

- Install ROS2 Humble from [here](#)
- If you are new to ROS2 there are useful tutorials on the page as well
- Next bash the following commands on the terminal:

```
1. Sudo apt update
2. sudo apt install ros-humble-desktop
3. sudo apt install ros-humble-urg-node
4. sudo apt install ros-humble-cartographer
5. sudo apt install ros-humble-cartographer-ros
6. sudo apt install ros-humble-nav2-map-server
7. sudo apt install libmodbus-dev
8. sudo apt install ros-humble-teleop-twist-joy
9. sudo apt install python3-pip
10. pip3 install evdev
```

- Install the following YP-Spur (BLVR-compatibility needed for I-Cart 3):

<https://github.com/BND-tc/yp-spur>

```
1. cd
2. git clone https://github.com/BND-tc/yp-spur.git
3. cd yp-spur
4. mkdir build
5. cd build
6. ../configure
7. make
8. sudo make install
```

- ▶ Install the YP-Spur – ROS2 bridge
- ▶ If you have not made a ROS2 workspace yet, do it now with the following commands:

```
1. mkdir -p ~/ros2_ws/src  
2. cd ~/ros2_ws/src
```

- ▶ Then clone the repository and build:

```
1. git clone  
https://github.com/haruyama8940/icart_mini_driver_ros  
2.git  
2. cd ~/ros2_ws  
3. colcon build  
4. source install/setup.bash
```

SLAM configuration my_robot_slam.lua

- ▶ Bash:

```
1. cd  
2. nano my_robot_slam.lua
```

- ▶ Then copy the my_robot_slam lua code from repository https://github.com/nikopes/Autonomous_robot_files to that file

Cartographer configurations

▶ Bash:

```
1. cd
2. nano cartographer_launch.py
```

- ▶ Then copy the configurations from [here](#) and paste it to cartographer_launch.py
- ▶ Note that you need to change the cartographer_config_dir according to your system

```
# 1. Configuration Variables
use_sim_time = False
cartographer_config_dir = '/home/niksal'
configuration_basename = 'my_robot_slam.lua'
resolution = '0.05'
publish_period_sec = '1.0'
```

Create a parameter file for the robot

• Bash:

```
1. cd
2. nano iCart3_60W0803.param
```

- Then copy the iCart3 60W0803 parameters from [here](#) and paste it to iCart3_60W0803.param

Starting the robot

- ▶ When starting for the first time bash:

```
1. sudo ldconfig
```

- ▶ Terminal 1 (YP-Spur):

```
1. sudo ypspur-coordinator -d /dev/ttyUSB* --blvr -p  
~/iCart3_60W0803.param
```

- ▶ Terminal 2 (ROS2 driver):

```
1. source ~/ros2_ws/install/setup.bash  
2. ros2 run icart_mini_driver icart_mini_driver
```

- ▶ Terminal 3 (Transforms / TF):

- ▶ 1. sudo chmod 666 /dev/ttyACM*
2. source /opt/ros/humble/setup.bash
3. ros2 run tf2_ros static_transform_publisher 0.2 0 0 0 0 0
base_link laser & ros2 run tf2_ros
static_transform_publisher 0 0 0 0 0 0 base_footprint
base_link

- ▶ Terminal 4 (Controller):

```
1. source /opt/ros/humble/setup.bash  
2. ros2 launch teleop_twist_joy teleop-launch.py  
joy_config:='xbox'
```

- ▶ Terminal 5 (LiDAR):

```
1. sudo chmod 666 /dev/ttyACM0  
2. source /opt/ros/humble/setup.bash  
3. ros2 run urg_node urg_node_driver --ros-args -p  
serial_port:=/dev/ttyACM0 -p frame_id:=laser -p  
angle_min:=-2.09 -p angle_max:=2.09
```

- ▶ Terminal 6 (Cartographer):

```
1. source /opt/ros/humble/setup.bash  
2. ros2 launch ~/cartographer_launch.py
```

Starting the robot

- ▶ Terminal 7 (Visualization):
Bash:

```
1. source /opt/ros/humble/setup.bash
2. rviz2
```

Then add: laserscan, map, and odometry (optional), and set them to their respective topics (/scan, /map, /odom)

- ▶ Terminal 8 (Save map):

```
1. source /opt/ros/humble/setup.bash
2. ros2 run nav2_map_server map_saver_cli -f
~/my_map_name
```

- ▶ Drive the robot by pressing RB and moving the right joystick
- ▶ If the controls are inverted (Robot moves back when trying to go forward and moves left when trying to move right, etc...)

Bash:

```
1. nano
~/ros2_ws/src/icart_mini_driver_ros2/src/icart_mini_driver_node.cpp
```

Delete everything in the file above, then replace it with the icart_mini_driver_node.cpp code from

https://github.com/nikopes/Autonomous_robot_files

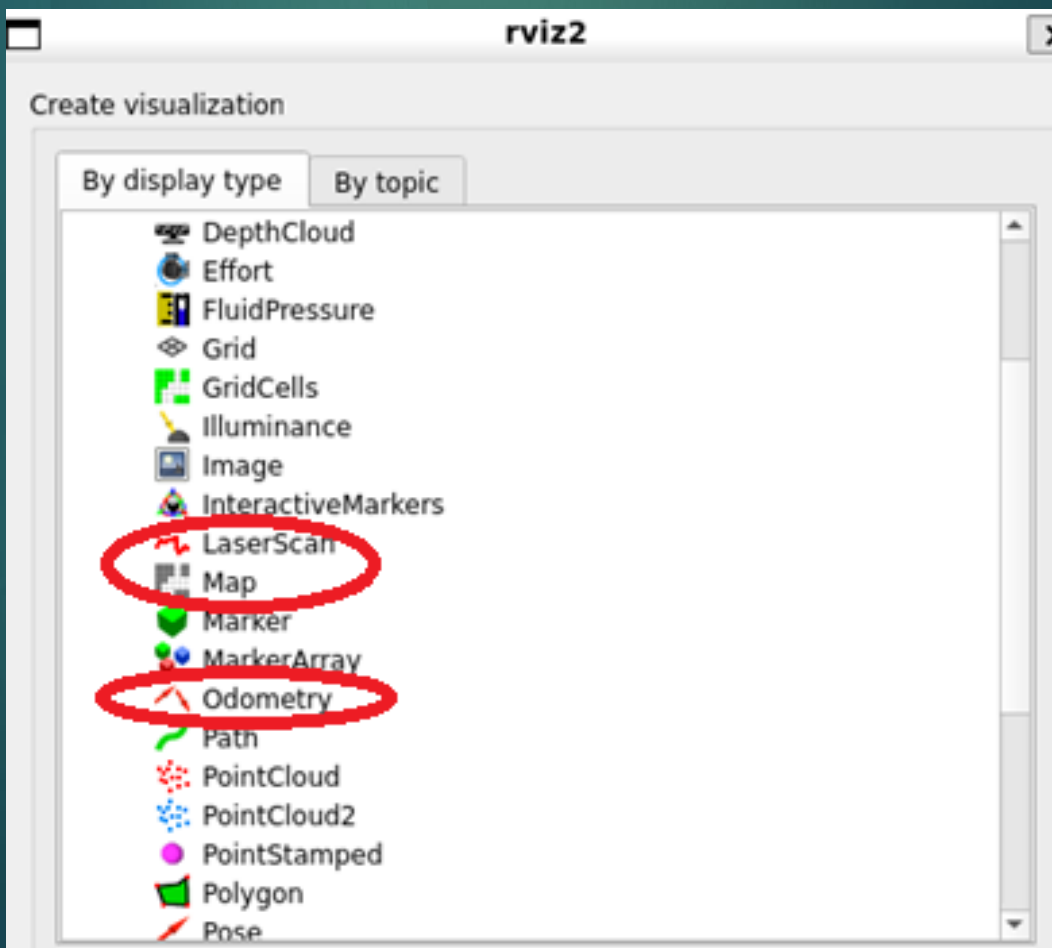
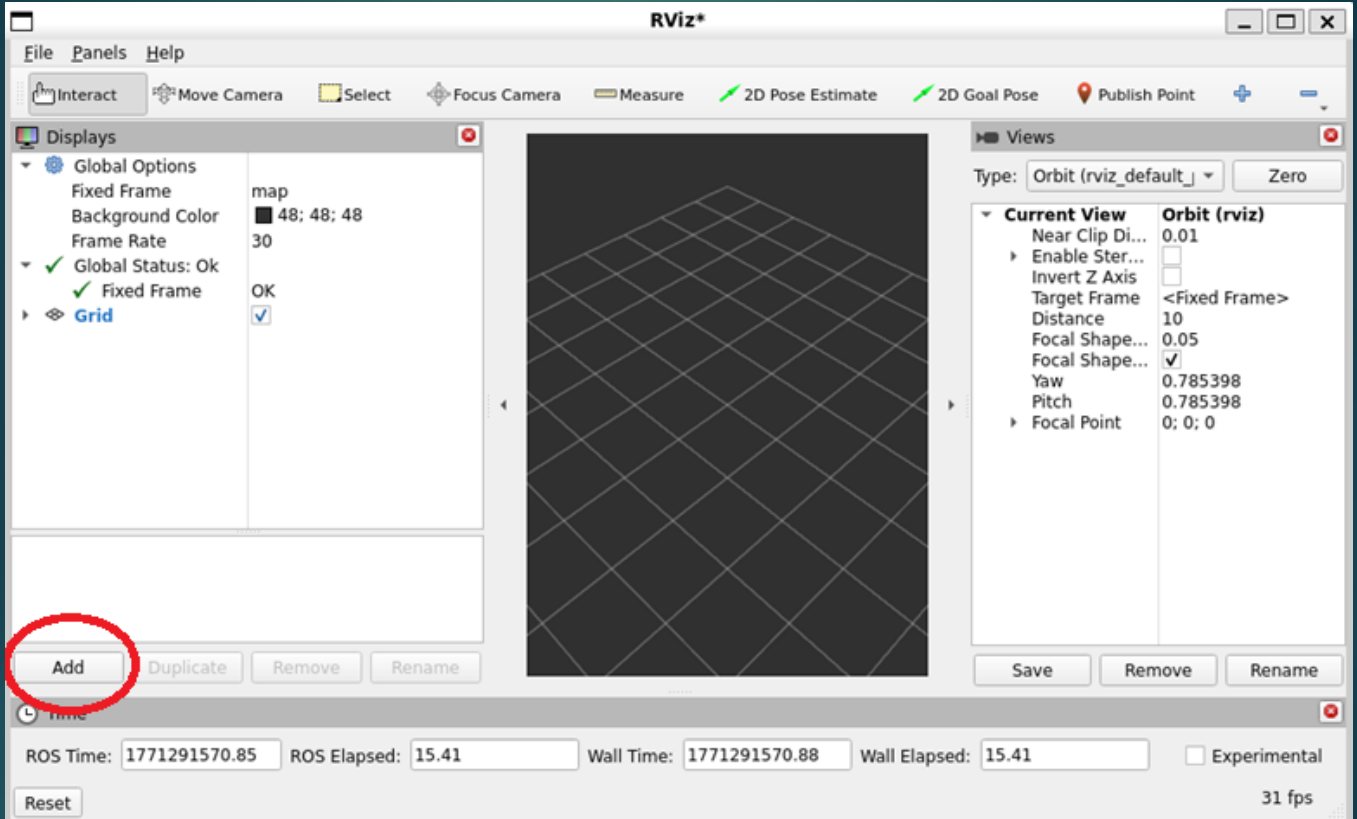
Then rebuild workspace:

```
1. cd ~/ros2_ws
2. colcon build
```

Now the controls should be correct!

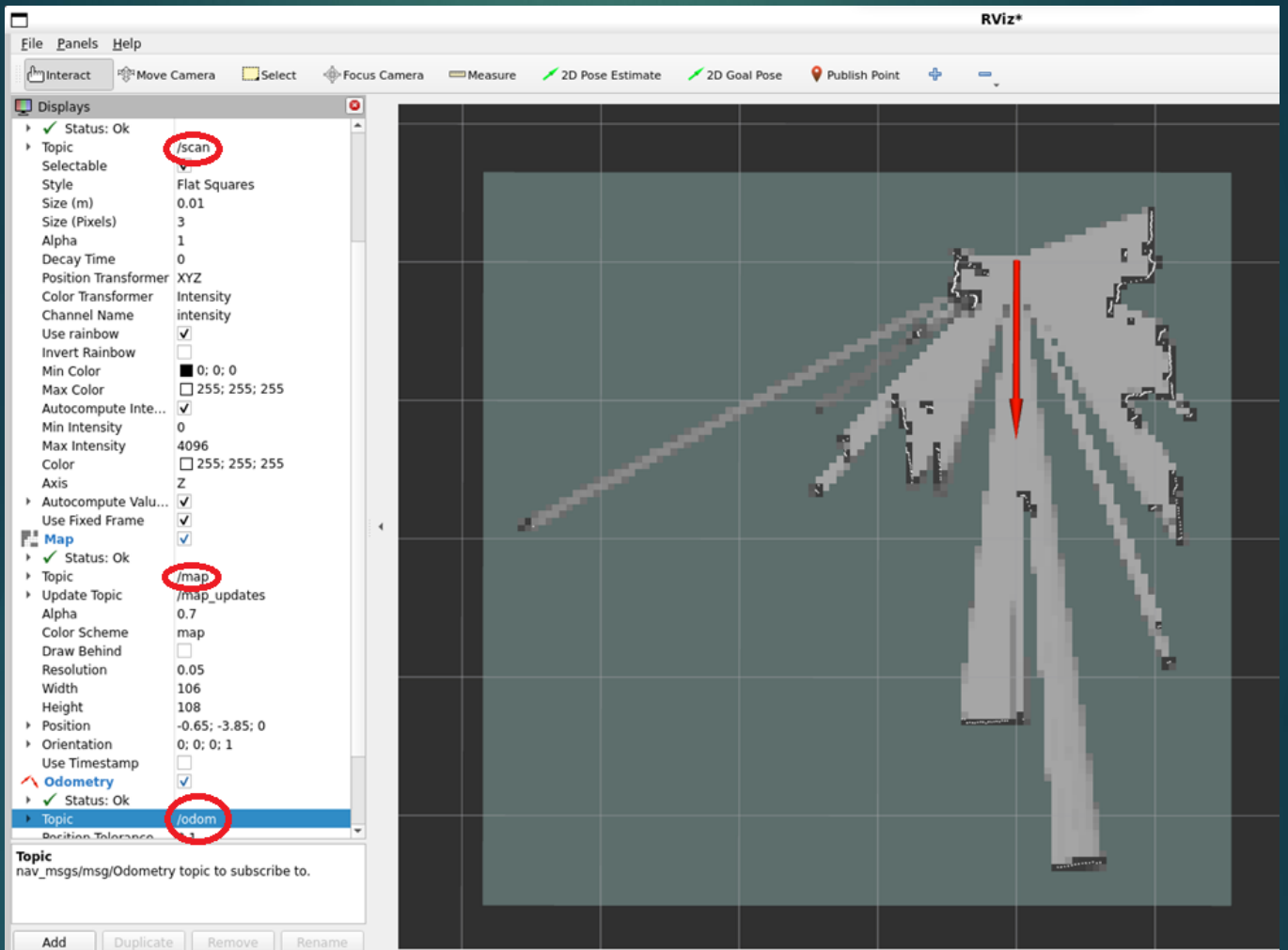
RVIZ configurations for SLAM

- First add LaserScan, Map and Odometry (Odometry is optional, but it shows the orientation of the robot on the map)



RVIZ configurations for SLAM

- ▶ Next select right topics for these
 - LaserScan = /scan
 - Map = /map
 - Odometry = /odom



How to open many terminals simultaneously

- ▶ Custom script is a good option.
Bash:

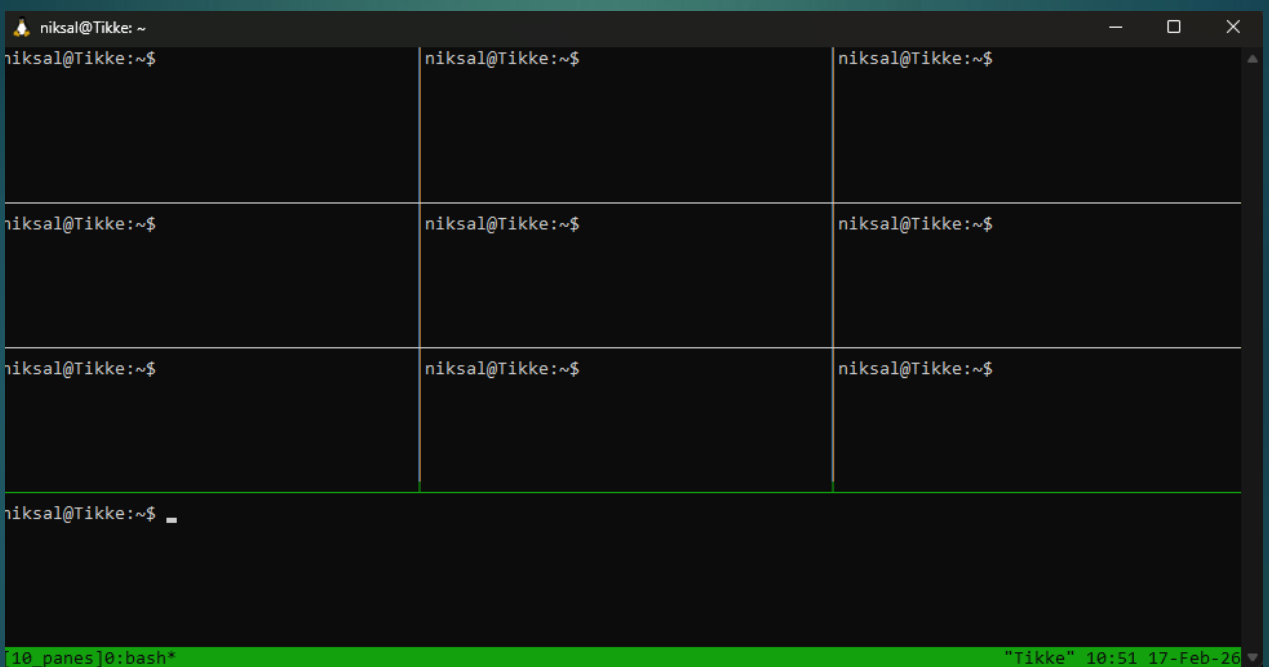
```
1. cd
2. nano open_10_panes.sh
```

- ▶ Copy the script from [here](#) to open_10_panes.sh
- ▶ Make the file executable by bashing:

```
1. chmod +x open_10_panes.sh
```

- ▶ You can now use the script by bashing:

```
1. ./open_10_panes.sh
```



- ▶ You can exit the 10 panes by bashing:

```
1. tmux kill-session -t 10_panes
```

Add a custom Navigation file

- ▶ The default navigation file is made for smaller, round robots like Turtlebot
- ▶ As Icart-3 is quite big, the parameters have to be modified
- ▶ I have prepared a pre-configured file with right size of the robot, and it is available [here](#)
- ▶ Use the following to create a config-directory to the icart_mini_driver folder, create the parameter file there and copy the code from the link above

```
1. mkdir -p ~/ros2_ws/src/icart_mini_driver_ros2/config
2. cp
/opt/ros/humble/share/nav2_bringup/params/nav2_
params.yaml
~/ros2_ws/src/icart_mini_driver_ros2/config/icart_nav
2_params.yaml
3. nano
~/ros2_ws/src/icart_mini_driver_ros2/config/icart_nav
2_params.yaml
```

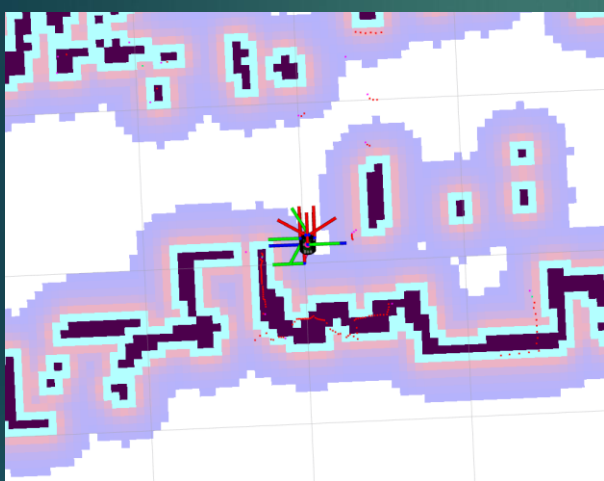
- ▶ Note that you might have to edit the file, max_vel_x, the cost scaling factors and inflation layer are very important
- ▶ Inflation layer is the size of the safety buffer around obstacles. If the robot hits objects, increase inflation_radius
- ▶ Cost scaling factor controls how far the robot tries to avoid obstacles. If the robot takes long paths, increase cost_scaling_factor

```
plugins: [ voxel_layer, inflation_layer ]
inflation_layer:
  plugin: "nav2_costmap_2d::InflationLayer"
  cost_scaling_factor: 3.0
  inflation_radius: 0.55
voxel_layer:
```

Effect of inflation radius and cost scaling factor

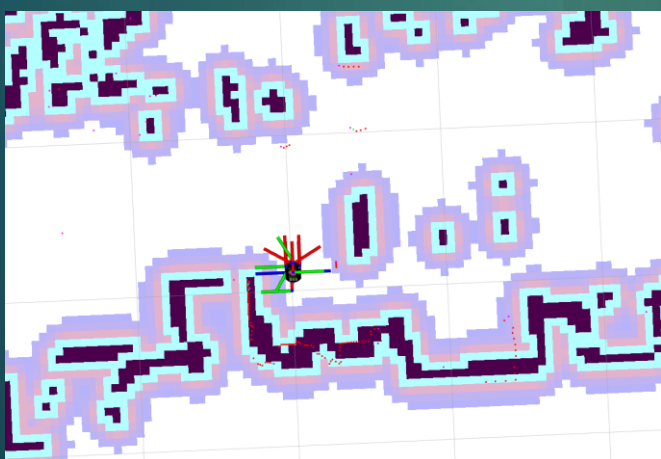
- Low cost_scaling factor and high inflation_radius make moving hard for the robot in small spaces

```
inflation_layer:  
  plugin: "nav2_costmap_2d::InflationLayer"  
  cost_scaling_factor: 15.0  
  inflation_radius: 0.30
```



- This can be improved with tuning the cost_scaling_factor up and inflation radius down

```
inflation_layer:  
  plugin: "nav2_costmap_2d::InflationLayer"  
  cost_scaling_factor: 25.0  
  inflation_radius: 0.16
```



- The perfect values depend on the robot and the environment, and are only found with careful tuning

Navigation

► Terminal 1:

```
1. source /opt/ros/humble/setup.bash
2. sudo ypspur-coordinator -d /dev/ttyUSB* --
blvr -p ~/iCart3_60W0803.param
```

► Terminal 2:

```
1. source ~/ros2_ws/install/setup.bash
2. ros2 run icart_mini_driver icart_mini_driver
```

► Terminal 3:

```
1. sudo chmod 666 /dev/ttyACM*
2. source /opt/ros/humble/setup.bash
3. ros2 run urg_node urg_node_driver --ros-args -p
serial_port:=/dev/ttyACM* -p frame_id:=laser -p
angle_min:=-2.09 -p angle_max:=2.09
```

► Terminal 4:

```
1. source /opt/ros/humble/setup.bash
2. ros2 run tf2_ros static_transform_publisher 0.2 0 0 0
0 0 base_link laser & ros2 run tf2_ros
static_transform_publisher 0 0 0 0 0 0 base_footprint
base_link
```

► Terminal 5:

```
1. source /opt/ros/humble/setup.bash
2. ros2 launch nav2_bringup bringup_launch.py
map:=/home/user/map_name.yaml
```

► Terminal 6:

```
1. source /opt/ros/humble/setup.bash
2. rviz2
```

> Add:

Laserscan /scan

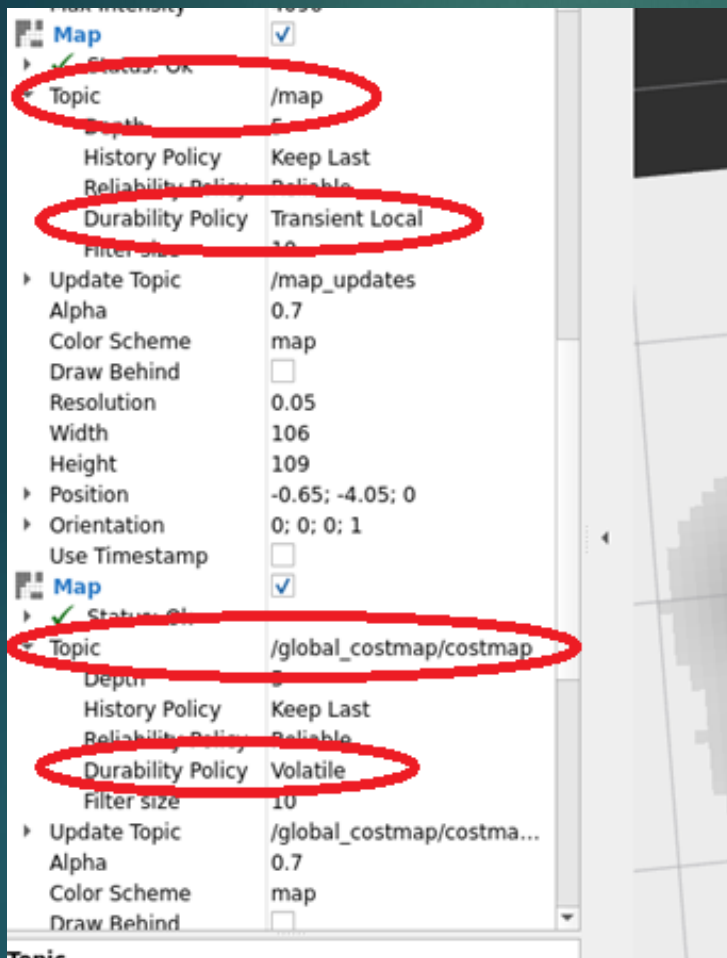
Map /map → Durability policy: transient local

Map /local_costmap

Map /global_costmap

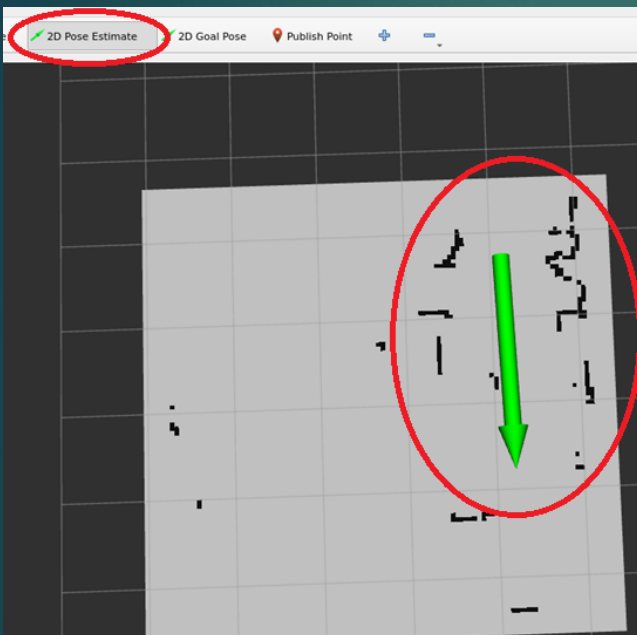
RVIZ configurations for navigation

- ▶ First add Map and LaserScan and set their topics accordingly
- ▶ Then add 2 other Map displays, and set their topics to /local_costmap and /global_costmap
- ▶ Set the transient local

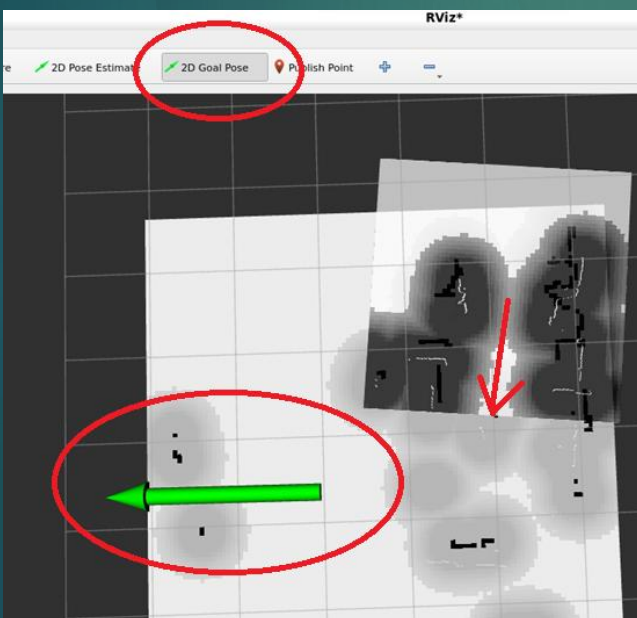


RVIZ configurations for navigation

- ▶ Set the 2D Pose Estimate based on where your robot is on the map in reality



- ▶ Set the 2D Goal Pose based on where you want your robot to navigate on the map



- ▶ The red arrow is the robot and direction it is looking at, and the black areas surrounding are local and global costmaps inflation layer discussed before